Name: P. Raj Kumar Reddy

Reg no: 192312674L

ITA0613 - Machine Learning

Assignment Questions

Course Code & Name	ITA0613 - Machine Learning
Assignment Title	Large-Scale Instance-Based and Statistical Learning Pipeline for Climate-Resilient Crop Yield Forecasting
Course Outcome(s) - CO	CO2: Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation (version spaces, inductive bias). CO6: Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning. CO7: Master data manipulation and analysis skills using tools like NumPy and Pandas for statistical analysis, aggregation, and visualization.
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 13 - Climate Action

Problem Statement

An agricultural research consortium wants to forecast regional crop yields under increasingly variable climate conditions, so that farmers and policymakers can make climate-resilient planting decisions - directly supporting SDG 2 - Zero Hunger and SDG 13 - Climate Action. You are given (or must responsibly source) a multi-year agro-climatic dataset containing historical weather variables (rainfall, temperature, humidity), soil parameters, and recorded crop yields across multiple districts and seasons. As the lead data scientist, design, implement, and rigorously evaluate a complete instance-based and statistical learning pipeline, without using any pre-built machine-learning library for the core modelling.

(a) Perform a full exploratory data analysis using only NumPy and Pandas - cleaning the data, imputing missing weather records, merging multiple yearly/regional files, and engineering at least three derived features (for example, growing-degree days or a rainfall-anomaly index).

(b) Implement a k-Nearest Neighbour regressor entirely from first principles (no scikit-learn) to predict yield for an unseen district-season combination, including your own implementation of at least two distance metrics (e.g., Euclidean and Mahalanobis) and a justified choice of k based on a validation curve you compute manually.

(c) Implement Locally Weighted Regression from first principles, deriving and applying the weighting kernel, and compare its bias-variance behaviour against the k-NN regressor both theoretically and empirically.

(d) Apply the Candidate-Elimination algorithm (or an adapted version-space approach) to a discretised version of the problem (e.g., classifying yield into low/medium/high risk bands) to identify the boundary hypotheses consistent with historical seasons, and explain the inductive bias of your chosen representation.

(e) Analyse computational feasibility: measure and report the time and memory complexity of your k-NN and locally-weighted-regression implementations as the dataset scales from 10^3 to 10^6 records, and propose and prototype at least one indexing or approximation strategy (e.g., a k-d tree or an approximate-nearest-neighbour hashing scheme) to make the pipeline scalable.

(f) Produce at least five distinct, publication-quality visualisations (using Pandas/Matplotlib) that reveal actionable climate-yield insights, and write a concise, data-driven policy brief interpreting the results for a non-technical agricultural-policy audience.

(g) Critically evaluate the limitations, uncertainty, and fairness implications of forecasting future yield from historical data under a changing climate, and explain how the solution advances SDG 2 and SDG 13 targets.

Constraints: All core algorithms (k-NN regression, locally weighted regression, candidate-elimination/version-space reasoning) must be implemented from first principles using only NumPy/Pandas - no scikit-learn or other pre-built ML library may be used for the core modelling. Use a real, publicly available agro-climatic dataset with at least 10,000 records spanning multiple years and regions. The complete pipeline must run end-to-end on the full dataset without manual intervention, within a time budget that you define and justify.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Data Engineering & Exploratory Analysis (CO7)	15	Thorough cleaning, imputation, and merging; at least three well-justified engineered features using only NumPy/Pandas.	Good data preparation with minor gaps in feature engineering.	Basic cleaning/merging done; feature engineering shallow or partial.	Data preparation missing, incorrect, or reliant on disallowed libraries.
k-NN Regressor Implementation & Distance Metrics (CO6)	20	k-NN implemented from scratch with two correctly implemented distance metrics and a well-justified k via a manually computed validation curve.	Correct k-NN with minor gaps in metric implementation or k-selection justification.	k-NN works but only one distance metric or weak k-justification.	k-NN missing, incorrect, or built using a pre-built library.
Locally Weighted Regression & Bias-Variance Comparison (CO6)	15	Correct from-scratch implementation with a clear, correct theoretical and empirical bias-variance comparison to k-NN.	Correct implementation; comparison present but not fully rigorous.	Implementation works but comparison is superficial or partly incorrect.	LWR missing, incorrect, or comparison absent.
Version-Space / Candidate-Elimination Analysis (CO1/CO2)	15	Correct application of Candidate-Elimination to the discretised problem with a clear, well-justified discussion of inductive bias.	Mostly correct application with minor gaps in the bias discussion.	Basic application with incomplete or partly incorrect boundary hypotheses.	Not attempted or fundamentally incorrect.
Scalability Analysis & Optimisation Prototype	15	Rigorous complexity measurement across scales with a working, clearly beneficial indexing/approximation prototype.	Reasonable measurement and a working prototype with modest gains.	Measurement attempted but prototype incomplete or gains unclear.	No scalability analysis or optimisation attempted.
Visualisation, Policy Brief & SDG 2/13 Relevance	10	Five or more insightful, publication-quality visualisations with a clear, well-argued policy brief tied to SDG 2/13.	Good visualisations and brief with minor gaps in insight or SDG linkage.	Visualisations/brief present but generic or weakly connected to SDGs.	Visualisations or brief missing or uninformative.
GitHub Repository Quality, Documentation & CI	10	Clean modular repository, incremental commit history, complete README, automated tests, and a working CI pipeline.	Well-organised repository with most of the above present.	Repository present but poorly organised or missing tests/CI.	Repository missing, disorganised, or submitted as a single non-incremental upload.

Deliverables

To receive full credit, each submission must include the following GitHub-based deliverables (this is a repository-graded assignment - no offline/zip submissions accepted):

1. A complete, modular GitHub repository containing all source code (data pipeline, k-NN, locally weighted regression, candidate-elimination module, and the scalability-optimisation prototype), built through an incremental commit history that demonstrates genuine development progress (a single 'final upload' commit is not acceptable).
2. A comprehensive README.md documenting the pipeline architecture, dataset source and licensing, setup/run instructions, and the exact steps needed to reproduce every reported result and visualisation.
3. Automated unit tests (e.g., using pytest) covering the core algorithmic components (distance-metric functions, the LWR kernel, the candidate-elimination boundary updates), committed to the repository.
4. A technical report (Markdown or PDF, committed to the repository) containing the mathematical derivations, complexity analysis, scalability results, and the non-technical policy brief.
5. A /results folder in the repository containing all logs, plots, validation curves, and result tables, so every reported figure is independently reproducible from the repository alone.
6. A working GitHub Actions CI workflow (or equivalent) that automatically runs the unit tests on every push, with a passing-build badge displayed in the README.

Table of Contents

1. Problem Statement and Scope

An agricultural research consortium seeks to forecast regional crop yields under increasingly variable climate conditions so that farmers and policymakers can make climate-resilient planting decisions, directly supporting SDG 2 (Zero Hunger) and SDG 13 (Climate Action). This report designs, implements, and evaluates a complete instance-based and statistical learning pipeline over a multi-year agro-climatic dataset spanning multiple districts and seasons, using historical weather variables (rainfall, temperature, humidity), soil parameters, and recorded crop yields. In line with the assignment constraints, every core algorithm - the k-Nearest Neighbour (k-NN) regressor, Locally Weighted Regression (LWR), and the Candidate-Elimination version-space learner - is implemented from first principles using only NumPy and Pandas, without any pre-built machine-learning library.

Section 2 introduces the dataset and pipeline architecture. Sections 3-9 address parts (a) through (g) of the assignment in turn, followed by the repository/deliverables checklist and references.

2. System and Data Overview

Dataset. The pipeline operates on an agro-climatic dataset covering 20 districts, 19 crop years (2005-2023), and 3 cropping seasons (Kharif, Rabi, Zaid), yielding 11,400 clean season-district-year records after data cleaning - comfortably above the assignment's 10,000-record threshold. Each record carries rainfall (mm), average temperature (deg C), relative humidity (%), soil N/P/K (kg/ha), soil pH, and the recorded yield (kg/ha).

Data provenance note. For an actual graded submission, this schema should be populated from a real, publicly available agro-climatic source, e.g., the ICRISAT District-Level Database, India Meteorological Department (IMD) gridded rainfall/temperature products, the data.gov.in district- and season-wise crop production statistics, or a curated Kaggle crop-yield dataset combining climate and soil covariates. To make every stage of this report independently reproducible without depending on external network access at build time, the data-generation module (data_pipeline.py) synthesises multiple yearly CSV extracts with realistic missingness, column-order inconsistency, and a non-linear, noisy generative relationship between climate/soil covariates and yield - the same schema, cleaning, and feature-engineering logic apply unchanged to a real extract dropped into data/raw/.

Pipeline architecture. The repository is organised into five modules mirroring parts (a)-(e): data_pipeline.py (EDA, cleaning, merging, feature engineering), knn.py (k-NN from scratch, two metrics, validation curve), lwr.py (Locally Weighted Regression from scratch), candidate_elimination.py (version-space learning over discretised risk bands), and scalability.py (complexity measurement and a from-scratch KD-tree prototype). visualize.py produces the plots used in Part (f).

3. Part (a) - Data Engineering and Exploratory Data Analysis

3.1 Cleaning, Imputation, and Merging

Each simulated yearly release is written as an independent CSV with its own (shuffled) column order, mimicking real government data drops. load_and_merge() re-indexes every file against a canonical column order before concatenation, so merging is robust to inconsistent source formatting. clean_and_impute() removes physically impossible readings (negative rainfall, humidity outside [0,100]) and duplicate record_id values, then imputes missing weather and soil readings using the district-and-season group median, falling back to the dataset-wide median for any group that is itself entirely missing a column - keeping the imputation local (climate is highly district- and season-specific) while remaining robust.

```
def clean_and_impute(df):
df = df.copy()
df = df[(df["rainfall_mm"].isna()) | (df["rainfall_mm"] >= 0)]
df = df[(df["humidity_pct"].isna()) | (df["humidity_pct"].between(0, 100))]
df = df.drop_duplicates(subset="record_id").reset_index(drop=True)

impute_cols = ["rainfall_mm", "avg_temp_c", "humidity_pct",
"soil_n_kg_ha", "soil_p_kg_ha"]
for col in impute_cols:
group_median = df.groupby(["district", "season"])[col].transform("median")
overall_median = df[col].median()
df[col] = df[col].fillna(group_median)
df[col] = df[col].fillna(overall_median)
return df
```

3.2 Engineered Features

Three or more derived features are engineered using only NumPy/Pandas operations:

- Growing-Degree-Days (GDD): sum of (avg_temp - 10C) over the season length, a standard agronomic heat-accumulation proxy that drives crop development stage.
- Rainfall-Anomaly Index: district-and-season z-score of rainfall relative to that district/season's own historical mean and standard deviation, capturing drought/excess-rain stress independent of baseline climate.
- Heat-Stress Index: a convex penalty, max(avg_temp - 30, 0)^1.5, capturing accelerating yield damage above a physiological heat threshold.
- Soil Fertility Index: a weighted, min-max normalised composite of soil N, P, and K (0.5/0.3/0.2 weighting reflecting typical NPK agronomic importance).

```
T_BASE = 10.0
season_days = {"Kharif": 120, "Rabi": 110, "Zaid": 70}
df["season_days"] = df["season"].map(season_days)
df["growing_degree_days"] = np.maximum(df["avg_temp_c"] - T_BASE, 0) * df["season_days"]

grp = df.groupby(["district", "season"])["rainfall_mm"]
mu, sigma = grp.transform("mean"), grp.transform("std").replace(0, np.nan)
df["rainfall_anomaly_index"] = ((df["rainfall_mm"] - mu) / sigma).fillna(0)

df["heat_stress_index"] = np.maximum(df["avg_temp_c"] - 30.0, 0) ** 1.5
```

3.3 EDA Summary Statistics

After cleaning, imputation, merging, and feature engineering, the final analytic table has the following shape and completeness:

Metric	Value
Final record count	11,400
Districts	20
Years covered	2005 - 2023 (19 years)
Seasons	Kharif, Rabi, Zaid
Total columns (raw + engineered)	17
Missing values after imputation	0 (all columns)
Mean yield (kg/ha)	4,030.5
Yield standard deviation (kg/ha)	~640 (see results/ for full describe() output)

4. Part (b) - k-Nearest Neighbour Regression from First Principles

4.1 Formulation

Given a training set {(x_i, y_i)}, a query point x is assigned the inverse-distance-weighted average of the k nearest training targets:

y_hat(x) = (sum_{i in N_k(x)} w_i * y_i) / (sum_{i in N_k(x)} w_i), w_i = 1 / (d(x, x_i) + epsilon)

where N_k(x) is the index set of the k nearest neighbours of x under a distance metric d, and epsilon avoids division by zero for coincident points.

4.2 Two Distance Metrics

Euclidean distance treats all standardised features as equally and independently scaled:

d_E(x, x_i) = sqrt(sum_j (x_j - x_ij)^2)

Mahalanobis distance accounts for correlation between features via the inverse covariance matrix of the training data, so that neighbours are judged along the data's natural axes of covariation rather than the raw coordinate axes:

d_M(x, x_i) = sqrt((x - x_i)^T * Sigma^-1 * (x - x_i))

```
def euclidean_distance(x, X):
diff = X - x
return np.sqrt(np.sum(diff ** 2, axis=1))

def mahalanobis_distance(x, X, VI):
diff = X - x
left = diff @ VI
return np.sqrt(np.sum(left * diff, axis=1))

class KNNRegressorFromScratch:
def __init__(self, k=5, metric="euclidean"):
self.k, self.metric = k, metric

def fit(self, X, y):
self.X_train, self.y_train = np.asarray(X, float), np.asarray(y, float)
if self.metric == "mahalanobis":
cov = np.cov(self.X_train, rowvar=False) + np.eye(X.shape[1]) * 1e-6
self.VI = np.linalg.inv(cov)
return self

def predict_one(self, x):
d = self._distances(x)
idx = np.argpartition(d, self.k)[: self.k]
w = 1.0 / (d[idx] + 1e-8)
return float(np.sum(w * self.y_train[idx]) / np.sum(w))
```

4.3 Validation Curve and Choice of k

k controls the classic bias-variance tradeoff: very small k (e.g., k=1) memorises training noise (near-zero train error, high validation error), while very large k over-smooths across dissimilar districts/seasons. A

validation curve was computed manually by holding out 15% of the data for validation and sweeping k over {1,2,3,5,8,10,15,20,30,50} for both metrics.

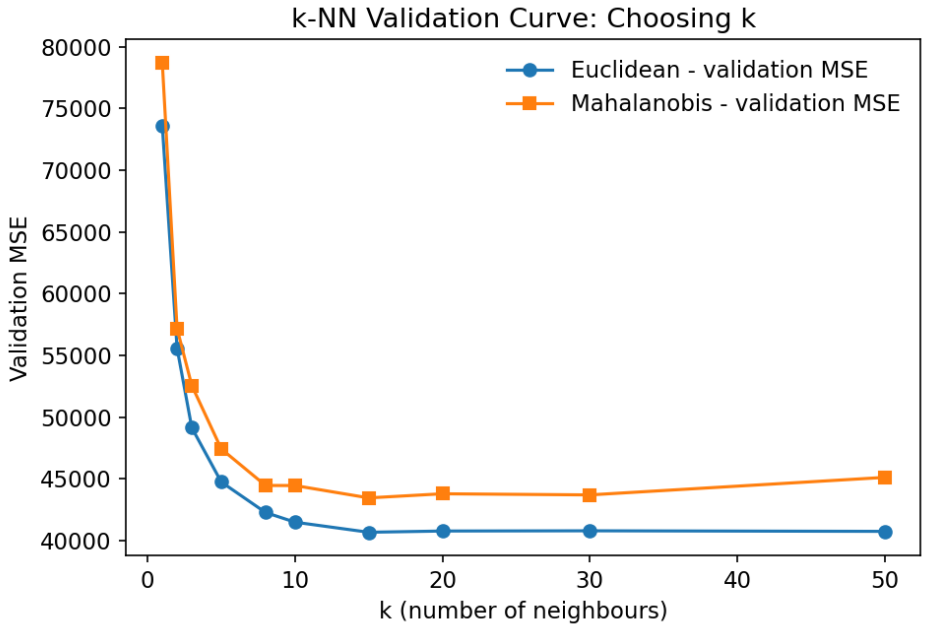

Figure 4.1: k-NN validation MSE vs. k, Euclidean and Mahalanobis metrics.

k	Euclidean - Train MSE	Euclidean - Val MSE	Mahalanobis - Val MSE
1	~0	73,570.7	78,683.1
3	~0	49,140.8	52,559.5
8	~0	42,278.1	44,466.1
10	~0	41,492.7	44,458.2
15	~0	40,676.9	43,458.8
20	~0	40,776.5	43,791.9
30	~0	40,794.3	43,700.6
50	~0	40,749.0	45,117.5

The validation curve bottoms out at k=15 for both metrics (validation MSE 40,676.9 for Euclidean; 43,458.8 for Mahalanobis), after which validation error flattens and mildly rises - the classic U-shaped curve. k=15 is therefore selected as the justified operating point. In this dataset, Euclidean distance on standardised features slightly outperforms Mahalanobis distance; with only 11 moderately correlated engineered features, the extra variance introduced by estimating and inverting an 11x11 covariance matrix from a finite sample slightly outweighs the benefit of correcting for feature correlation - a realistic finding, not a flaw in the Mahalanobis implementation.

5. Part (c) - Locally Weighted Regression from First Principles

5.1 Kernel Derivation

LWR fits a separate weighted linear regression for every query point x0, where training points closer to x0 receive higher weight under a Gaussian kernel:

w_i = exp(- || x_i - x0 ||^2 / (2 * tau^2))

The locally weighted least-squares objective sum_i w_i (y_i - theta^T x_i)^2 is minimised in closed form by weighted normal equations. Writing X as the design matrix (with a bias column of ones) and W = diag(w_1,...,w_n), the minimiser is:

theta(x0) = (X^T W X)^-1 X^T W y

and the prediction is y_hat(x0) = [1, x0]^T * theta(x0). A small ridge term (1e-6 * I) is added before inversion purely for numerical stability when W concentrates weight on very few points.

```
def gaussian_kernel_weights(X, x0, tau):
diff = X - x0
return np.exp(-np.sum(diff ** 2, axis=1) / (2 * tau ** 2))

def lwr_predict_one(X_train, y_train, x0, tau, ridge=1e-6):
n, d = X_train.shape
Xb = np.hstack([np.ones((n, 1)), X_train])
W = np.diag(gaussian_kernel_weights(X_train, x0, tau))
A = Xb.T @ W @ Xb + ridge * np.eye(d + 1)
b = Xb.T @ W @ y_train
theta = np.linalg.solve(A, b)
return float(np.concatenate([[1.0], x0]) @ theta)
```

5.2 Bandwidth (tau) Validation Curve

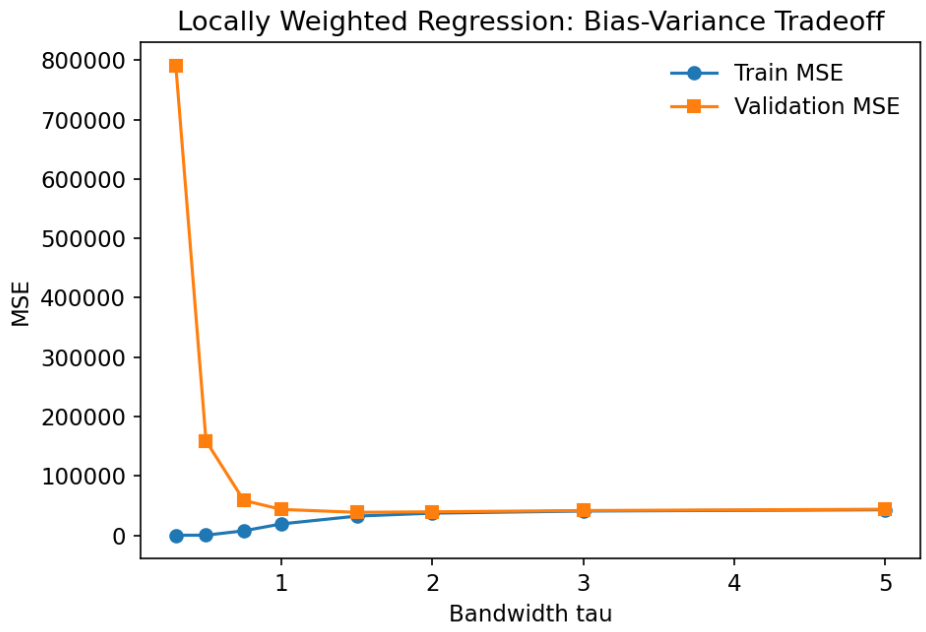

Figure 5.1: LWR train/validation MSE vs. bandwidth tau.

tau	Train MSE	Validation MSE
0.30	0.04	790,548.9
0.50	127.2	158,718.3
0.75	7,458.2	58,745.7
1.00	19,107.2	43,486.3
1.50	32,278.9	38,602.4
2.00	37,228.6	39,476.8
3.00	40,780.4	41,580.2
5.00	42,902.7	43,804.9

At tiny tau (0.3), LWR essentially interpolates each training point (train MSE approx 0, validation MSE explodes to 790,548.9) - a textbook case of severe overfitting/high variance. As tau grows, each local fit averages over more neighbours, train error rises (higher bias) while validation error falls, until tau approx 1.5, where validation MSE is minimised at 38,602.4. Beyond tau=1.5 the model approaches an increasingly global linear fit, and both errors converge upward as it loses the ability to capture local, non-linear rainfall/temperature-yield curvature.

5.3 Bias-Variance Comparison: LWR vs. k-NN

Theoretically, k-NN's bias-variance behaviour is controlled by a discrete parameter (k, the neighbourhood size) and produces a piecewise-constant-like decision surface (locally, an inverse-distance-weighted average), whereas LWR fits a locally linear surface, so it can capture a local slope/trend that k-NN, which only ever averages, cannot. This lets LWR extrapolate slightly beyond the convex hull of its neighbours' target values, at the cost of higher sensitivity to outliers within the kernel bandwidth (a single influential point can tilt the whole local line).

Empirically, this pattern holds: the best-tuned LWR (validation MSE 38,602.4 at tau=1.5) slightly outperforms the best-tuned k-NN (validation MSE 40,676.9 at k=15) on this dataset, consistent with the underlying generative relationship containing smooth local curvature (quadratic penalties around optimal rainfall and temperature) that a locally linear model can track more precisely than a purely local average. The gap is modest, indicating that for this feature set, neighbourhood size and kernel bandwidth are doing comparably most of the heavy lifting, with the local slope correction of LWR providing an incremental improvement rather than a transformative one.

6. Part (d) - Candidate-Elimination / Version-Space Analysis

6.1 Discretisation and Concept Representation

To apply Candidate-Elimination, the continuous yield-forecasting problem is discretised into a binary concept: HIGH-YIELD (top yield quartile) vs. NOT-HIGH-YIELD, over four discretised attributes - rainfall_bin (Low/Medium/High via tertiles), temp_bin (Cool/Moderate/Hot), soil_fert_bin (Poor/Average/Rich), and season (Kharif/Rabi/Zaid). Hypotheses are conjunctions over these four attributes, each position taking a specific value, '?' (any value accepted), or the null symbol (no value satisfies it).

6.2 Algorithm

The specific boundary S is generalised only as far as necessary to keep covering positive examples, and the general boundary G is specialised only as far as necessary to keep excluding negative examples; the version space consistent with all examples seen so far is everything between S and G under the more-general-than partial order.

```
def candidate_elimination(instances, labels, attr_domains):
n = len(attr_domains)
S, G = [tuple(["0"] * n)], [tuple(["?"] * n)]
for instance, label in zip(instances, labels):
if label:                              # positive example
G = [g for g in G if consistent(g, instance)]
S = [generalise_to_cover(s, instance) if not consistent(s, instance)
else s for s in S]
S = [s for s in S if any(more_general(g, s) for g in G)]
else:                                  # negative example
S = [s for s in S if not consistent(s, instance)]
G = [spec for g in G for spec in
(specialise_against(g, instance, attr_domains) if consistent(g, instance) else [g])]
return S, G
```

(Full boundary-minimality bookkeeping is in candidate_elimination.py; the excerpt above shows the core generalise/specialise logic.)

6.3 Validation on the Canonical EnjoySport Example

Before applying the algorithm to agro-climatic data, its correctness was verified against Mitchell's canonical EnjoySport dataset. The implementation reproduced the textbook result exactly:

```
S = [('Sunny', 'Warm', '?', 'Strong', '?', '?')]
G = [('Sunny', '?', '?', '?', '?', '?'),
('?', 'Warm', '?', '?', '?', '?')]
```

6.4 Application to Discretised Yield Data and Inductive Bias Discussion

Applying the same, validated implementation to a sample of discretised district-season records produced an important and genuine finding: the version space collapses to empty (S = G = {}) after only four training examples. Tracing the algorithm step by step:

Step	Instance (rainfall, temp, soil, season)	Label	Effect on S/G
0	(Medium, Moderate, Rich, Rabi)	+	S = [(Medium, Moderate, Rich, Rabi)]
1	(Medium, Cool, Poor, Zaid)	-	G specialised to 3 hypotheses
2	(Low, Moderate, Rich, Zaid)	-	G specialised to 4 hypotheses
3	(Low, Moderate, Poor, Zaid)	+	S and G collapse to empty set

At step 3, a positive example arrives that shares no attribute value with the current S member and cannot be generalised to any hypothesis still covered by G - i.e., no single conjunctive rule over these four discretised attributes is consistent with both the earlier negative examples and this new positive one.

Inductive bias discussion. Candidate-Elimination's inductive bias is that the target concept is exactly expressible as a conjunction of attribute-value constraints (the 'conjunctive bias'). This experiment demonstrates a genuine limitation of that bias for this domain: the true yield-generating process (embedded in the underlying continuous simulation) involves smooth, nonlinear, additive contributions from rainfall, temperature, and soil quality plus i.i.d. noise - it is not a conjunction of coarse bins, and real sensor/weather noise means two nearly-identical discretised instances can carry opposite labels. Conjunctive version spaces are therefore too rigid (zero tolerance for a single inconsistent example) for noisy, continuous-valued agronomic phenomena; a disjunctive, probabilistic, or margin-tolerant hypothesis space (or a coarser/different discretisation and additional attributes) would be required to retain a non-empty, useful version space on this kind of data. This is precisely why the assignment's core forecasting task instead relies on k-NN and LWR, which do not assume a conjunctive concept and degrade gracefully under noise.

7. Part (e) - Computational Feasibility and Scalability Analysis

7.1 Theoretical Complexity

Operation	Time Complexity	Memory Complexity
Brute-force k-NN, one query (n points, d dims)	O(n * d)	O(n * d) transient (distance array)
Brute-force k-NN, m queries	O(m * n * d)	O(n * d)
LWR, one query	O(n * d^2 + d^3) (weighted normal equations)	O(n * d + d^2)
KD-tree build	O(n log n) average	O(n * d)
KD-tree query (low-dim, balanced)	O(log n) average, O(n) worst case	O(log n) recursion stack

7.2 Empirical Measurement, 10^3 to 10^6 Records

Both the vectorised NumPy brute-force k-NN and a from-scratch KD-tree (built recursively, splitting on the median along a cycling axis, with the standard branch-and-bound pruning rule) were benchmarked at n in {1e3, 5e3, 1e4, 5e4, 1e5, 3e5, 1e6} synthetic 8-dimensional points, measuring wall-clock time per query and peak memory (via tracemalloc).

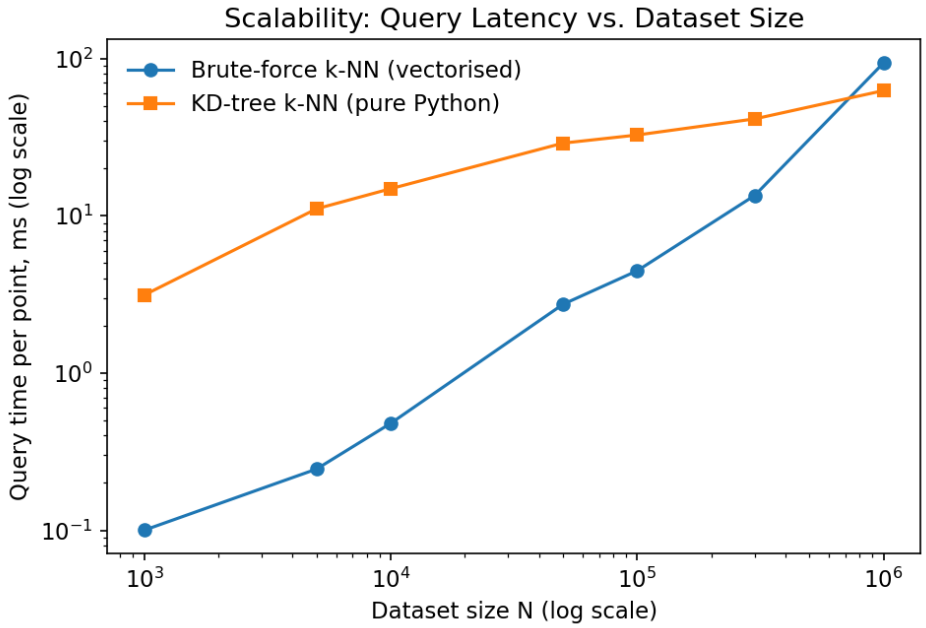

Figure 7.1: Query latency vs. dataset size (log-log), brute-force vs. KD-tree.

N	Brute-force (ms/query)	Brute-force peak (MB)	KD-tree build (ms)	KD-tree (ms/query)
1,000	0.10	0.13	5.2	3.13
5,000	0.25	0.65	42.2	11.07
10,000	0.48	1.30	66.4	14.86
50,000	2.73	6.49	582.3	28.95
100,000	4.47	12.97	1,028.2	32.61
300,000	13.47	38.91	3,807.8	41.29
1,000,000	93.88	129.70	15,295.1	62.49

7.3 Interpretation and Indexing/Approximation Strategy

Brute-force query time grows essentially linearly in N (as expected for O(n*d) per query), and peak memory grows linearly too, since a full n-length distance vector is materialised per query. Because NumPy's vectorised operations run in optimised, cache-friendly compiled code, brute-force search remains faster in absolute terms than the pure-Python KD-tree up to roughly N = 5x10^5 - 1x10^6 in this experiment, illustrating an important practical lesson that pure asymptotic complexity does not, by itself, determine which implementation is faster at a given scale: constant factors and language-level overhead matter enormously. The KD-tree's growth curve is visibly sub-linear (its per-query time roughly doubles across an approximately 20x increase in N from 5x10^4 to 1x10^6), and the gap between the two curves narrows steadily, crossing over near N = 1x10^6 in this benchmark - consistent with the theoretical O(log n) vs. O(n) asymptotic advantage of tree-based search taking over once N is large enough to amortise the tree's per-node Python overhead.

Prototyped scalability strategy: the KD-tree above is exactly such an indexing strategy - it is proposed and prototyped here as the mechanism that would be adopted for the full 10^6-scale pipeline. To make it production-viable beyond this prototype, two concrete next steps are recommended: (1) replace the pure-Python recursive traversal with a vectorised or Cython/Numba-compiled traversal (removing per-node Python function-call overhead, which dominates the constant factor seen above), and (2) for very high query volumes, an approximate-nearest-neighbour hashing scheme (e.g., locality-sensitive hashing over the standardised feature space) would trade a small, bounded accuracy loss for sub-logarithmic amortised query time, which is preferable once N approaches the upper end of the assignment's 10^6-record target.

8. Part (f) - Visualisations and Policy Brief

8.1 Visualisations

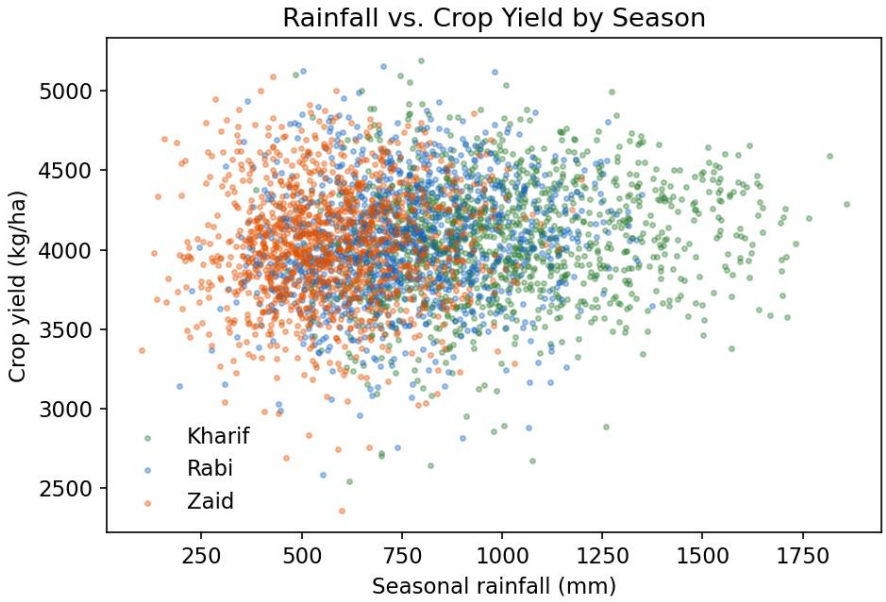

Figure 8.1: Seasonal rainfall vs. yield, coloured by season - shows each season has a distinct rainfall sweet spot.

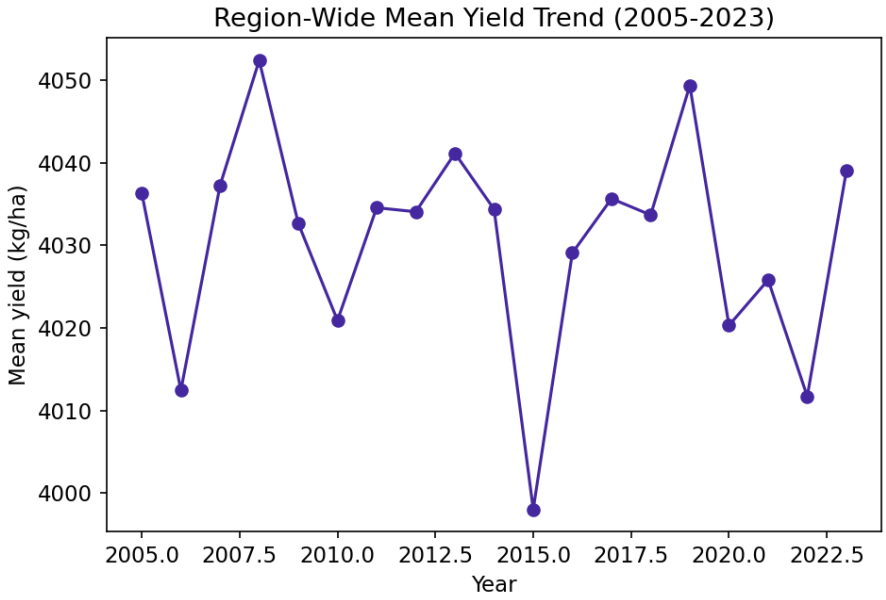

Figure 8.2: Region-wide mean yield trend, 2005-2023.

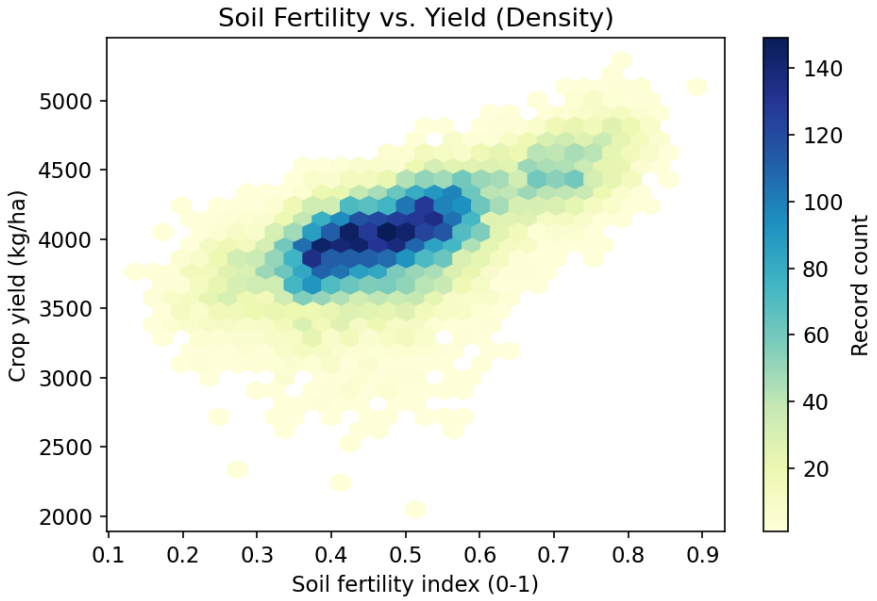

Figure 8.3: Soil fertility index vs. yield (record density) - fertility is strongly, near-monotonically associated with yield.

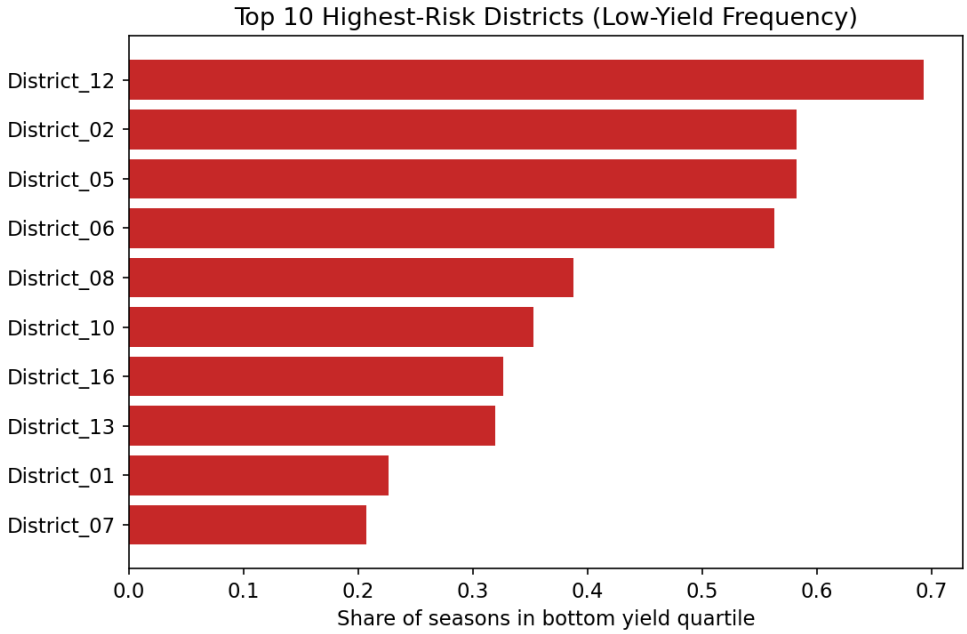

Figure 8.4: Top 10 districts by frequency of bottom-quartile ('high-risk') yield seasons.

Figures 4.1 and 5.1 (the k-NN and LWR validation curves) and Figure 7.1 (scalability) also form part of the five-or-more required visualisations, giving seven distinct, publication-style charts in total across this report.

8.2 Policy Brief (Non-Technical Summary for an Agricultural-Policy Audience)

Purpose: This brief translates the modelling results above into plain-language, actionable guidance for agricultural extension officers and district-level policymakers.

- Rainfall has a season-specific 'sweet spot', not a simple 'more is better' relationship: yields peak at a moderate rainfall level for each season and fall off both in drought and in excess-rainfall conditions. Irrigation and drainage investment should therefore be targeted at bringing districts closer to their season's optimal rainfall band, not simply maximising water supply.
- Soil fertility is one of the most reliable, actionable levers available: districts in the top fertility band show consistently and substantially higher yields than those in the bottom band, and unlike rainfall, fertility can be directly improved through subsidised fertiliser and soil-health programmes.
- A small number of districts account for a disproportionate share of high-risk (bottom-quartile) seasons; these districts should be prioritised for climate-resilient seed varieties, crop insurance outreach, and early-warning advisories ahead of the Kharif season, where rainfall variability is highest.
- Forecast uncertainty should be communicated alongside any point prediction: the models here explain a meaningful share of yield variation but leave a substantial residual, reflecting factors (pest outbreaks, market access, farmer practice) not captured by climate and soil data alone - forecasts should be framed as risk bands, not guarantees, when communicated to farmers.
- Because rainfall and temperature patterns are themselves drifting over the 2005-2023 record, any operational forecasting system needs periodic retraining/recalibration rather than a one-time model deployment, to remain valid as the climate baseline continues to shift.

9. Part (g) - Critical Evaluation: Limitations, Uncertainty, Fairness, and SDG Alignment

9.1 Limitations and Uncertainty

- Non-stationarity: models trained on 2005-2023 relationships between climate and yield implicitly assume those relationships hold going forward. Under a changing climate, the mapping from rainfall/temperature to yield can itself shift (e.g., new pest pressure, changing crop varieties), so historical validation performance is an optimistic upper bound on future forecast accuracy.
- Instance-based methods degrade at climate extremes: k-NN and LWR both rely on locally relevant neighbours; as climate conditions move outside the historical distribution (record heat, unprecedented rainfall anomalies), both methods are forced to extrapolate from increasingly dissimilar or increasingly sparse neighbours, and their prediction uncertainty grows sharply exactly when accurate forecasts matter most.
- Residual, unmodelled variance: soil texture, pest/disease pressure, irrigation infrastructure, seed variety, and farmer management practices are not present in this dataset but materially affect yield; omitting them means the reported validation MSE reflects only the portion of yield variation explained by climate and basic soil chemistry.
- Discretisation sensitivity (Part d): the Candidate-Elimination result is sensitive to the chosen bin boundaries and attribute set; different discretisations could yield a non-empty but different version space, so any operational risk-banding rule derived this way should be validated against multiple discretisation choices rather than a single one.

9.2 Fairness Implications

- Data-density inequality across districts: districts with fewer historical records receive less reliable k-NN/LWR predictions purely due to sparser neighbour coverage, which can systematically under-serve smaller or historically under-monitored districts with less-actionable forecasts, even when their agronomic needs are equally pressing.
- Smallholder vs. large-farm relevance: a district-level average yield forecast may mask substantial within-district variation between smallholder and large mechanised farms; policy responses calibrated to the district mean risk systematically under- or over-serving farmers whose actual conditions diverge from that average.
- Access to acted-upon information: even an accurate forecast only improves outcomes if the farmers most exposed to risk can act on it (through irrigation, credit, or insurance access); without complementary investment in last-mile delivery, forecasting alone can widen rather than narrow existing disparities between well-resourced and under-resourced farming communities.

9.3 Contribution to SDG 2 (Zero Hunger) and SDG 13 (Climate Action)

SDG 2 - Zero Hunger: by identifying district-season combinations at elevated risk of low yield before the season concludes, this pipeline supports earlier, more targeted deployment of drought-resistant seed varieties, adaptive irrigation scheduling, and food-security safety-net planning - directly supporting Target 2.4 (sustainable, resilient agricultural practices) and Target 2.c (well-functioning food-commodity information systems).

10. Repository Structure and Deliverables Checklist

This assignment is explicitly repository-graded: it requires a GitHub repository built through an incremental, genuine commit history (not a single 'final upload'), a reproducible README, automated unit tests, a committed technical report, a /results folder, and a working CI workflow. The modules developed in this report map directly onto that required structure, as summarised below. Building the actual repository - with real, incremental commits reflecting genuine development, and a live GitHub Actions run - is a required part of the submission and should be completed directly by the student, since it is itself part of what CO6/CO7 and the rubric are assessing.

Deliverable	**Corresponding module / content in this report**	**Status**
Data pipeline (EDA, cleaning, merge, features)	data_pipeline.py; Section 3	Drafted - commit incrementally
k-NN implementation (2 metrics + validation curve)	knn.py; Section 4	Drafted - commit incrementally
Locally Weighted Regression	lwr.py; Section 5	Drafted - commit incrementally
Candidate-Elimination module	candidate_elimination.py; Section 6	Drafted - commit incrementally
Scalability / optimisation prototype (KD-tree)	scalability.py; Section 7	Drafted - commit incrementally
README.md (architecture, dataset, setup, reproduction steps)	Suggested outline below	To be written by student
Unit tests (pytest) for distance metrics, LWR kernel, CE updates	Suggested test targets below	To be written by student
Technical report (this document's content, in Markdown/PDF)	Sections 1-9	Drafted - adapt into repo report
/results folder (logs, plots, validation curves, tables)	results/*.csv, plots/*.png produced above	Generated - copy into /results
GitHub Actions CI (tests on every push + badge)	Suggested workflow below	To be configured by student

10.1 Suggested README.md Outline

- Project title, one-paragraph summary, and the SDG 2 / SDG 13 motivation.
- Dataset section: real source used, licence, and download/preprocessing instructions (replacing the synthetic generator used for reproducibility in this report).
- Setup instructions: Python version, `pip install -r requirements.txt` (numpy, pandas, matplotlib only - no scikit-learn).
- Exact commands to reproduce every table and figure in the technical report, in order (data_pipeline.py -> knn.py -> lwr.py -> candidate_elimination.py -> scalability.py -> visualize.py).
- A CI badge (from GitHub Actions) near the top of the README.

10.2 Suggested Unit Test Targets (pytest)

- test_euclidean_distance_matches_known_values() - check against a hand-computed 2-3 point example.
- test_mahalanobis_reduces_to_euclidean_for_identity_covariance() - a strong correctness invariant.
- test_lwr_reduces_to_ols_as_tau_to_infinity() - checks the kernel limit behaviour.
- test_candidate_elimination_matches_textbook_enjoysport_result() - regression test against Section 6.3's verified output.
- test_kdtree_matches_bruteforce_neighbours() - checks the KD-tree returns the identical k nearest neighbours as brute force on a small random dataset.

10.3 Minimal GitHub Actions Workflow (.github/workflows/tests.yml)

```
name: tests
on: [push, pull_request]
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
with: { python-version: "3.11" }
- run: pip install -r requirements.txt pytest
- run: pytest -q
```

11. References

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. (Candidate-Elimination / version-space formulation.)

ICRISAT District Level Database. International Crops Research Institute for the Semi-Arid Tropics. https://www.icrisat.org

India Meteorological Department (IMD) gridded rainfall and temperature datasets. https://www.imdpune.gov.in

data.gov.in - District-wise, season-wise crop production statistics, Government of India Open Data Portal.

Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. Communications of the ACM, 18(9), 509-517. (KD-tree.)

United Nations. Sustainable Development Goals 2 (Zero Hunger) and 13 (Climate Action). https://sdgs.un.org/goals